Trabalho Prático 03 - Algoritmos I

7 de junho de 2025

1 Introdução

Clara adora jogar xadrez, e gosta de inventar desafios diferentes com as peças do jogo. Dessa vez, ela gostaria de uma peça que se movimenta com maior liberdade, então escolheu a rainha.

O jogo que ela propôs é o seguinte: há uma rainha e alguns peões em um tabuleiro personalizado. Ela quer utilizar a rainha para capturar todos os peões no mínimo de movimentos possível.

A rainha pode se movimentar como da forma tradicional em um tabuleiro de xadrez, ortogonalmente e diagonalmente. Para simular os outros aspectos de um tabuleiro, há espaços que funcionam como se tivessem outras peças, sendo intransponíveis para a rainha.

2 Descrição do Problema

A primeira linha contém três valores: N e M, as dimensões do tabuleiro, e K, o número de peões no tabuleiro. As próximas N linhas contém M caractéres e representam a disposição do tabuleiro, com as seguintes representações:

- '.' é uma célula vazia
- 'R' é a célula inicial da rainha
- 'P' é uma célula que contém um dos peões
- '-' é uma célula intransponível

As limitações são que $4 \leq N,\ M \leq 100$ e $1 \leq K \leq 16$.

2.1 Resposta

Você deve responder o **número mínimo de movimentos necessários para capturar todos os K peões**. Se isso não for possível, a saída deve ser -1.

2.2 Algorítmos

Para esse último TP, diferentemente dos anteriores, a solução deverá ser implementada de duas formas diferentes. Em uma delas, é necessária a utilização de um algoritmo exato, garantindo a resposta correta para o problema. A outra implementação deve ser mais eficiente e pode dar uma resposta aproximada. **Ambas as implementações devem constar na documentação, e a comparação entre elas é IMPRESCINDÍVEL**.

3 Solução

3.1 Modelagem

Os algoritmos utilizados podem compartilhar da mesma modelagem geral do problema, mas devem diferir na forma de encontrar o número mínimo de movimentos. Vale ressaltar que, para a resposta aproximada, é permitido o uso de heurísticas, mas a documentação deverá apresentar a forma como se chega na resposta. Também é válido lembrar que, para a aproximação, não há um algorítmo certo esperado, mas é crucial que a documentação, análise de complexidade e o algorítmo implementados sejam correspondentes.

3.2 Entrada e Saída

Nessa sessão está especificado o que será enviado como entrada e o que é esperado como saída da sua solução. Serão disponibilizadas VPL **apenas para a solução exata**. Abaixo segue o formato generalizado:

Entrada:

```
N M K
A00A01[...]A0M
A10A11[...]A1M
[...]
AN0AN1[...]ANM
```

Saída:

```
S
```

3.3 Exemplos

3.3.1 Exemplo 01

Entrada:

```
4 4 1
...R
....
....
P...
```

Saída:

```
1
```

3.3.2 Exemplo 02

Entrada:

```
4 4 1
...R
--..
P-..
....
```

Saída:

```
3
```

3.4 Exemplo 03

Entrada:

```
8 6 3
......
P-....
..R..-
.-....
..-...
.-...-
.P...P
-.-..-
```

Saída:

```
4
```

3.5 Exemplo 04

Entrada:

```
6 6 3
.-----
-.--P.
--.--.
-.-.-.
P-.-.R
.....P
```

Saída:

```
6
```

3.6 Discussão

Dado a necessidade de dois algoritmos, parte do trabalho será desenvolver uma discussão, comparando o tempo e qualidade das duas respostas. Serão disponibilizados 8 casos de teste no moodle para que o aluno execute em ambiente da sua escolha (fora da VPL) e meça, através do código (recomendamos o uso das bibliotecas padrão de tempo, **std::chrono** para C++ e **time** para Python). É importante apresentar a resposta de ambos os algoritmos e o tempo gasto para cada um dos casos de teste.

4 Implementação

4.1 Linguagem

O trabalho prático deverá ser implementado em `C`, `C++` ou `Python` e utilizar apenas as bibliotecas padrão das respectivas linguagens. Não será permitido o uso de bibliotecas exclusivas de um sistema operacional ou compilador.

4.1.1 C

No caso de `C`, você deve usar apenas bibliotecas do padrão ANSI C, das versões `C99`, `C11` ou `C17`.

4.1.2 C++

No caso de `C++`, utilize bibliotecas do padrão `ISO/IEC C++`, das versões `C++11`, `C++14`, `C++17` ou `C++20`.
Poderão ser utilizadas bibliotecas que implementam algumas funcionalidades mais básicas, como:

- Algumas estruturas de dados simples: `<vector>`, `<set>`, `<list>`, etc.
- Entrada e saída: `<iostream>`, `<fstream>`, etc.
- Manipulação de strings: `<string>`, `<sstream>`, etc.
- Algumas utilidades simples: `<utility>`, `<compare>`, etc.

Não poderão ser utilizadas bibliotecas que realizam funcionalidades de mais alto nível, como:

- Algoritmos mais complexos: `<algorithm>`, `<functional>`, `<ranges>`, etc.
- Gerenciamento automático de memória: `<memory>`, `<memory_resource>`, etc.

Em caso de dúvidas, pergunte aos monitores.

4.1.3 Python

No caso de Python, serão aceitas versões feitas em Python 3.9+, com acesso a todas as bibliotecas padrão da linguagem. Pacotes adicionais não serão permitidos, e eles não funcionarão na VPL. Em caso de erro do código devido a importação de bibliotecas proibidas, ele NÃO SERÁ AVALIADO.

4.2 Ambiente

O aluno pode implementar em qualquer ambiente de programação que desejar, mas deve garantir que o programa seja compilável nas máquinas do DCC, que são acessíveis aos alunos e disponibilizadas pelo Centro de Recursos Computacionais (para mais informações, acesse o site: https://www.crc.dcc.ufmg.br/). A maneira mais fácil de conferir isso é rodando o código nos VPL's de entrega, caso tudo compile corretamente, é um forte indicativo de que o programa está compatível com o ambiente das máquinas do DCC.

4.3 Código

Utilize boas práticas de programação que você aprendeu em outras disciplinas, para que seu código seja legível e possa ser interpretado corretamente pelo leitor. A qualidade do seu código será avaliada. Algumas dicas úteis:

- Seja consistente nas suas escolhas de indentação, formatação, nomes de variáveis, funções, estruturas, classes e outros.
- Escolha nomes descritivos e evite nomear variáveis como `aux`, `tmp` e similares.
- Comente o seu código de forma breve e objetiva para descrever funções, procedimentos e estruturas de dados, mas evite comentários muito longos e de várias linhas.
- Para c,c++, separe seu código em diferentes arquivos, como `.c` e `.h` para C ou `.cpp` e `.hpp` para C++, de forma que facilite navegar pelo seu código e compreender o fluxo de execução.
- Para o Python, não é necessária a utilização de arquivos de headers.
- Evite funções muito grandes ou muito pequenas, que fazem várias coisas diferentes ao mesmo tempo, ou que tenham ou retornem muitos parâmetros diferentes.

4.4 Compilação

4.4.1 C,C++

Ao compilar o programa, você deverá utilizar no mínimo as seguintes flags:

```
-Wall -Wextra -Wpedantic -Wformat-security -Wconversion -Werror
```

Se seu programa apresentar erros de compilação, seu código não será corrigido.

4.4.2 Python

Para Python, por ser uma linguagem interpretada, o código não precisa ser compilado. Ainda assim, ele não deve apresentar erros em sua execução

4.5 Parâmetros

O aluno deve ler o arquivo de entrada do programa pela entrada padrão através de linha de comando, como por exemplo:

```
./tp3 < testCase01.txt
```

E gerar o resultado na saída padrão, não por arquivo.

5 Documentação

O aluno deverá fornecer uma documentação do trabalho contendo as seguintes informações:

1. **Introdução:** Uma breve explicação, em suas palavras, sobre qual é o problema computacional a ser resolvido.
2. **Modelagem:** Como você modelou o problema, traduzindo a situação fictícia em uma estrutura de dados, e quais algoritmos foram utilizados.
3. **Solução:** Como os algoritmos que você implementou resolvem o problema proposto e qual a ideia geral de cada um deles. A explicação deve ser de alto nível e/ou utilizar pseudocódigo, sem trechos diretos do código fonte.
4. **Análise de Complexidade:** Para cada um dos três problemas deve haver uma analíse da complexidade assíntotica demonstrada e explicada de tempo e memória da solução escolhida.
5. **Discussão dos Algorítmos:** Apresentar as respostas e tempos de execução dos algorítmos, e dissertar sobre a diferença, comparando a perda de qualidade com o aumento de eficiência.
6. **Referências:** Liste aqui as referências que você utilizou, considerando aquilo que foi relevante para a resolução deste trabalho prático.

A documentação deverá ser **sucinta e direta**, explicando com clareza o processo, contendo não mais do que 5 páginas.

6 Entrega

A entrega deverá ser feita através do Moodle, sendo um arquivo `.zip` ou `.tar.gz` no formato `MATRICULA_NOME`, contendo os seguintes itens:

- A documentação em um arquivo `.pdf`;
- Um arquivo `Makefile` que crie um executável com o nome `tp3`, (desnecessário para resoluções em Python);
- Todos os arquivos de código fonte.

Ademais, também serão disponibilizados VPL's para a correção automática, para que você receba a pontuação pela corretude do código é necessário enviar sua solução para esses VPL's. **Apenas a solução exata deve ser enviada na VPL**.

7 Correção

Seu trabalho prático será corrigido de forma automática, e portanto, você deverá garantir que, ao rodar o comando `make` na pasta contendo os arquivos extraídos, seja gerado um binário executável com o nome `tp3`, para que seu código seja avaliado corretamente.

Serão avaliados casos de teste básicos, bem como casos mais complexos e específicos, que testarão não somente a corretude, mas também a performance da sua solução. Para que seu código seja avaliado, você deverá garantir que seu programa dê a resposta correta e ótima, conforme pedido na descrição dos problemas. Esses casos serão disponibilizados no Moodle para que você possa testar seu programa.

Você deve garantir que seu programa não apresente erros de execução ou vazamentos de memória. Caso seu programa apresente erros de execução em algum caso de teste, sua nota será zerada para o caso específico. Vazamentos de memória serão penalizados.

Em caso de suspeita de plágio, seu trabalho será zerado e os professores serão informados. É importante fazer o trabalho por conta própria e não simplesmente copiar a solução inteira de outra pessoa. Caso você tenha suas próprias ideias inspiradas em outras, deixe isso claro na seção de referências.

A entrega do código-fonte e da documentação é **obrigatória**. Na ausência de algum desses, seu trabalho não será corrigido, e portanto, será zerado.

8 Avaliação

A nota final (NF) do trabalho prático será composta por dois fatores: o Fator Parcial de Implementação (FPI) e o Fator Parcial de Documentação (FPD).

8.1 Fator Parcial de Implementação (FPI)

A implementação será avaliada com base nos seguintes aspectos:

Aspecto	Sigla	Valores Possíveis
Compilação no ambiente de correção	co	0 ou 1
Respostas corretas nos casos de teste	ec	0 a 100%
Tempo de execução abaixo do limite	te	0 ou 1
Qualidade do código	qc	0 a 100%

Tabela 1: Aspectos de avaliação da implementação

A fórmula para cálculo do FPI é:

$$FPI = co \times (ec - 0{,}15 \times (1 - qc) - 0{,}15 \times (1 - te))$$

Caso o valor calculado do FPI seja menor que zero, ele será considerado igual a zero.

8.2 Fator Parcial da Documentação (FPD)

A documentação será avaliada com base nos seguintes aspectos:

Aspecto	Sigla	Valores Possíveis
Apresentação (formato, clareza, objetividade)	ap	0 a 100%
Modelagem computacional	mc	0 a 100%
Descrição da solução	ds	0 a 100%
Análise de complexidade	ac	0 a 100%

Tabela 2: Aspectos de avaliação da documentação

A fórmula para cálculo do FPD é:

$$FPD = 0{,}4 \times mc + 0{,}4 \times ds + 0{,}2 \times ac - 0{,}25 \times (1 - ap)$$

Caso o valor calculado do FPD seja menor que zero, ele será considerado igual a zero.

8.3 Nota Final do Trabalho Prático (NF)

A nota final do trabalho prático será obtida pela equação:

$$NF = 10 \times (0{,}6 \times FPI + 0{,}4 \times FPD)$$

8.4 Atraso

Para trabalhos entregues com atraso, haverá uma penalização conforme a fórmula:

$$\Delta p = \frac{2^{(d-1)}}{64}$$

onde d representa a quantidade de dias de atraso. Assim, sua nota final será atualizada da seguinte forma:

$$NF := NF \times (1 - \Delta p)$$

Note que a penalização é exponencial e, após 7 dias de atraso, a penalização irá zerar a sua nota.

9 Considerações Finais

Leia atentamente a especificação e comece o trabalho prático o quanto antes. A interpretação do problema faz parte da modelagem. Em caso de dúvidas, procure os monitores, eles estão a disposição para solucionar dúvidas e ajuda-los.

Busque primeiro usar o fórum de dúvidas no Moodle, pois a dúvida de um geralmente é a dúvida de muitos.